1 Introduction

This report describes the design and implementation of a basic packet sniffing algorithm in C. The goal of the system is to capture network packets, analyse their headers, and detect specific malicious activities, including SYN flooding, ARP cache poisoning, and access to blacklisted URLs (Google and Facebook)[2]. The following sections talk about the system architecture, threading strategy, memory handling, and testing methodology.

2 Multithreading Architecture

2.1 Design Rationale

To handle high volumes of network traffic, a multithreaded approach was adopted. This allows multiple packets to be dealt with at once, and also different packets to be parsed on different cores of a multicore system instead of one packet at a time on one core. This higher rate of packets parsed means the program will run faster and allow the program to deal with high volumes of traffic without losing packets. The program uses a thread pool model, where a fixed number of worker threads process packets concurrently. This allows the main capture thread to remain responsive, reducing packet loss during peak traffic.

I used a first-in-first-out queue structure to handle the packets because it allows me to ensure fairness between each packet, as well as make processing predictable [1]. Since each packet (process) requires a very similar amount of processing, there is no need to implement a priority queue since each packet is of the same importance as the rest. The queue, combined with packet processing costs being similar for most packets, naturally provides balanced workload distribution across the thread pool. Each worker pulls packets independently, avoiding the need for more complex scheduling schemes and ensuring that no worker becomes idle while the queue is non empty.

The number of worker threads is fixed before the program starts running. This eliminates the extra processing needed for thread creation and destruction while also allowing for a multithreaded approach which can make use of all the cores in the system's CPU. A fixed sized pool to a relatively low number also avoids having too many threads at the same time which would require extra computation to switch between different packets without improving the actual performance of the program.

Since packets are sent on a network fairly frequently, a thread pool is adequate at handling continuous and unbounded packet arrival thanks to the queue. The separation of the main thread (capturing packets) to the worker threads (parsing and analysing the packets) means that a separate thread is constantly running and looking for incoming packets, and therefore packet data will not be lost even if all other threads are busy parsing packets.

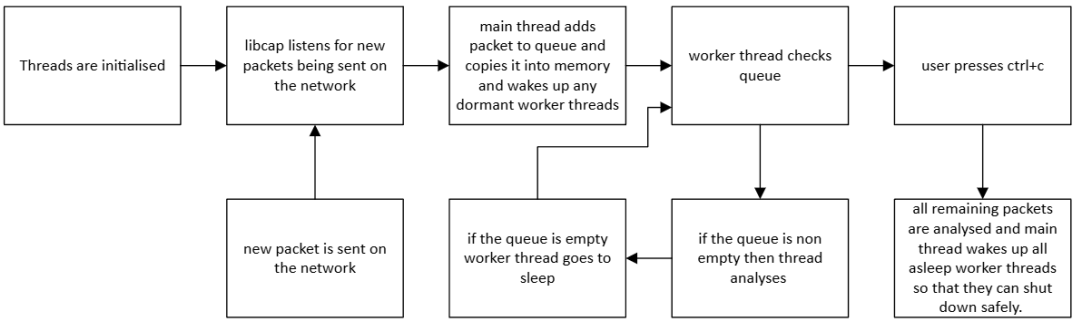


Figure 1: general flow chart of algorithm.

2.2 Thread Pool Structure

The main thread enqueues packets using the `dispatch()` function, which then creates a copy of that packet in memory and passes the copy onto the shared work queue. This queue consists of structs containing pointers in memory to the header, data and verbose flag.

Worker threads continuously dequeue packets and call the `analyse()` function to detect the malicious part of the packet.[1] Once the worker thread finishes parsing the packet, it frees up the memory used by the packet since the packet data was stored on the heap. This is because it is used in multiple parts of the program and by multiple functions and so we have to free the storage or we will get a memory leak. When the queue is empty, workers block on a condition variable called `queueNotEmpty`, waking only when new packets are available or when a shutdown signal is received to tell the worker function that the program is going to be terminated[3].

2.3 Dealing with Signals

The program uses signals to deal with certain key events within the program. The first being when the user presses ctrl+C to indicate to the program that they want to stop taking in new packets and output the result. However the specification also asks that the program still finished all current packets within the program so pressing Ctrl+C cannot simply terminate the program. To fix this i created a an instance of `SIGINT` which when turned on,it activated a boolean flag which was what indicated to the program that the user wanted the program to finish executing. This flag prevented new packets from being

added to the end of the queue.

The second signal is one within the Thread handler which was implemented through the `queueNotEmpty` condition variable. The signal is used by the Main thread to control access by the worker threads to the packet queue. The condition variable is used in two situations, the first being when a new packet is received by the main thread, `pthread_cond_signal()` is invoked to wake a single worker thread that is currently asleep since it is waiting for the queue to become non empty. The second case is when the program is going to be terminated, the main thread uses `pthread_cond_broadcast()` so that all dormant threads wake up and check that the program termination flag has been activated. Therefore the system can shut itself down while guaranteeing that no worker thread currently holds a mutex lock or is being indefinitely blocked while waiting on a non-empty queue.

2.4 Synchronisation Mechanisms and Memory Safety

Access to the packet queue is protected by a mutex to prevent race conditions. This is because a worker thread accessing the queue could interfere with the main thread pushing an item to the front of the queue. Condition variables are used to coordinate between the main and worker threads to indicate that there is a new packet in the queue, which could previously be empty, and therefore all threads should wake up so that one can process the new packet. A shutdown flag ensures that all threads terminate without errors when a SIGINT is received, and any remaining packets are processed before exit.

All packet headers and payloads are copied into memory when enqueued to avoid overwriting buffers from `libpcap` [4]. Each captured packet is deep-copied before being put in the queue, because `libpcap` reuses its internal buffer for all packets that come after. Without copying, worker threads would read corrupted or overwritten data and it would cause the output to be incorrect. Deep-copying therefore guarantees correct memory ownership and prevents use-after-free errors.

Worker threads free the memory associated with each packet after processing. Since the program wants a set of all the source IP addresses received for each SYN packet, I implemented a binary tree to store the IP addresses. Since we want to maintain the set properties within the tree, we have to query the tree if the IP already exists within the tree after every packet. In a tree, this can be done on average in O(log n) time, which is far better than O(n) in a list. Since we only ever add items or query the size, I believed this was the best option. I used Error handling, which ensures that packet capture failures or thread creation errors are logged and handled gracefully.

The variables that represent the number of SYN, ARP and blacklisted URLs are all stored within global variables. The problem with global variables is that they can be modified and accessed by any part of the program, including different worker threads. This can cause race conditions, which cause output data to be incorrect. To fix this, I created mutex locks for each set of global variables; one for ARP, SYN and URL. This not only keeps the different variables safe when a thread is working on them, but also, since each part has a different lock, it means that a SYN packet can be parsed completely in parallel to an ARP Packet without either thread needing to wait for a mutex lock.

3 Testing Methodology

3.1 SYN Flooding:

Tested using `hping3` to generate high-rate SYN packets to the loopback interface. Sent 100,000 packets with random IPs. This is to test both the system under a lot of traffic and that it is able to count the number of unique IPs, which should be roughly 100000. After running the test, I received the following result from my output:

3.1.1 Small scale controlled test

I then sent 10 packets from the same IP address so that it knows to recognise IP addresses it has seen before and to test that SYN detection work using this bash function in a different terminal `hping3 -c 10 -d 120 -S -w 64 -p 80 -i u100 localhost`.

```
workspace/skeleton/build/idsniff invoked. Settings:
        Interface: lo
        Verbose: 1
SUCCESS! Opened lo for capture
^C

Intrusion Detection Report:
10 SYN packets detected from 1 different IPs (syn attack)
0 ARP responses (cache poisoning)
0 URL Blacklist violations (0 google 0 facebook)
```

Figure 2: result of the program after sending the 10 packets.

This also proves the the SIGINT signal works since after the ^C is inputted, report is displayed which shows the program has terminated.

3.1.2 Randomised Source IP Test

I sent 100,000 packets from randomised IP addresses using this command : `hping3 -c 100000 -d 120 -S -w 64 -p 80 -i u10 --rand-source localhost` and saw the following result.

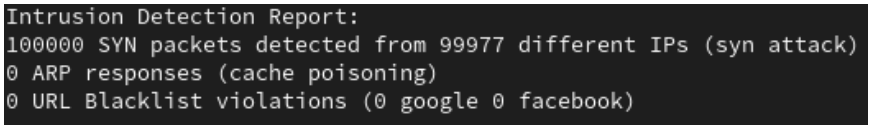


Figure 3: result of the program after sending the 100,000 SYN packets.

As we can see all 100000 packets were received and roughly 100000 different IP addresses were detected.

3.2 ARP Poisoning:

Validated with the provided `arp-poison.py` script to simulate unsolicited ARP replies. Repeated the function 3 times and here was the output

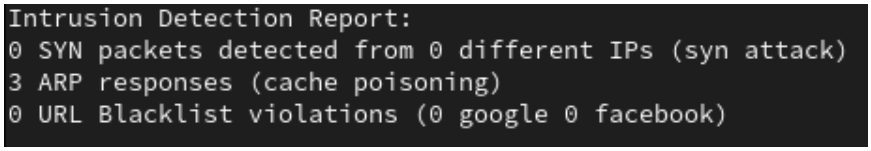


Figure 4: result of the program after sending the 3 ARP packets.

3.3 Blacklisted URLs:

Tested using `wget` requests to `www.google.co.uk` and `www.facebook.com` on the monitored interface (1 for each). Here was the result :

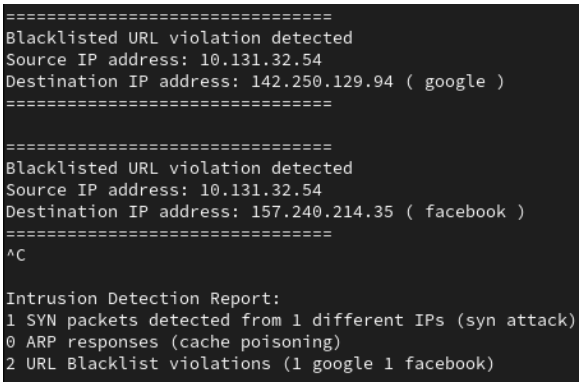


Figure 5: result of the program after sending the 2 packets with malicious URLS.

As we can see that each URL was detected and the final result was changed accordingly

3.4 Concurrency, Memory and Stress Tests

The program was tested under high traffic conditions to make sure the work queue and threads handled packet bursts without dropping packets or encountering race conditions. The program was also tested using Valgrind to make sure no memory leaks happened during the high traffic and this was the result of running the following function `www.google.co.uk` which constantly sent packets without delay using `hping3 --flood -d 120 -S -w 64 -p 80 localhost` . After 10 seconds i halted the program and Valgrind gave me the following output :

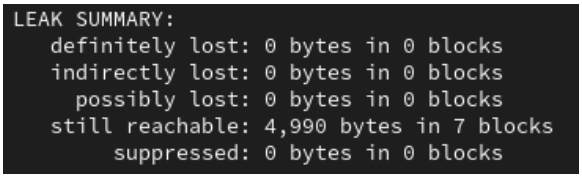


Figure 6: Valgrind output after 10 seconds of constantly sending packets.

This means that my code does not contain any memory leaks or segmentation faults.

4 Conclusion

The implemented program successfully captures and analyses network packets, detects specified malicious activities, and reports them in real-time. The use of the thread pool strategy allows efficient handling of high traffic while maintaining memory safety and correct synchronization. Testing confirmed the accuracy of detection mechanisms and the robustness of multithreaded operation. Future improvements could include making the binary tree containing IP addresses into an AVL tree.

References

[1] M. Aboelwafa, A. Abdelkader, and A. Hamza. A dynamic and scalable parallel network intrusion detection system. *Future Generation Computer Systems*, 124:152–166, 2021.

[2] F. Faraz. A simple packet sniffer in c using libpcap. `https://gist.github.com/fffaraz/7f9971463558e9ea9545`, 2014.

[3] Jianmin Wang, Dongbo Chen, and Yaqiang Zhang. Packet capture based on multi-core binding technology in linux. *Applied Mechanics and Materials*, 48:902–906, 2011.

[4] Shuguang Wen and Gaogang Xie. libpcap-mt: A multi-threaded packet capture library. In *Proceedings of the Network Operations and Management Symposium*, 2011.